

Análisis de Rendimiento y Escalabilidad: Motor QSimplify

1. Introducción y Objetivo

El propósito de este informe es documentar las pruebas de rendimiento (benchmarks) implementadas para medir los límites matemáticos y la escalabilidad del algoritmo de simplificación de grafos de QSimplify.

El objetivo principal fue determinar cómo el tiempo de ejecución es afectado por el incremento progresivo de complejidad (medida en número de qubits), estableciendo un límite de tiempo máximo aceptable de “5 minutos” (300 segundos) por procesamiento, tras el cual el algoritmo se considera inoperativo.

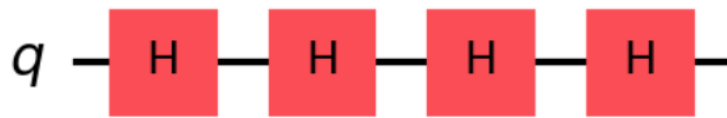
2. Metodología de Pruebas y Casos de Estrés

Para someter el sistema a estrés, las pruebas se diseñaron basándose en un único parámetro de crecimiento: “El número de qubits”. Se iteró progresivamente desde 1 hasta 8 qubits evaluando los siguientes tres comportamientos críticos:

(Nota: Los diagramas presentados a continuación son representaciones visuales a escala reducida... durante las pruebas de rendimiento reales, estos patrones fueron multiplicados masivamente (generando hasta 100 compuertas por qubit) para inyectar una carga de estrés significativa y evaluar los límites reales del motor.)

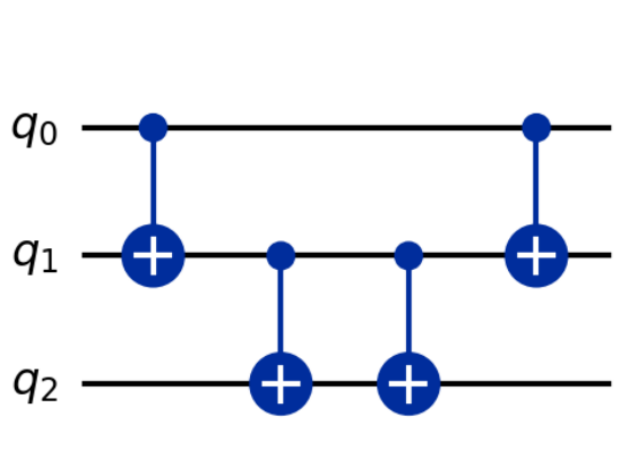
Caso 1: Cadenas de Anulación Simple (Carga lineal)

Propósito: Medir la eficiencia del algoritmo para identificar y eliminar redundancias masivas en componentes aislados .



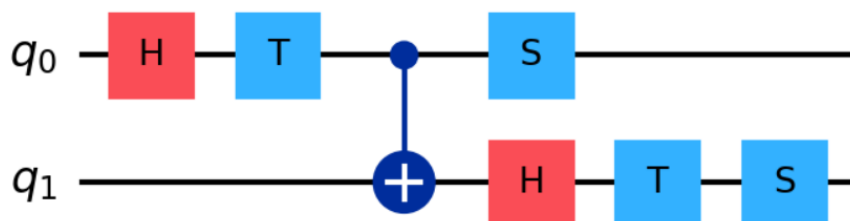
Caso 2: Cascada de CNOTs (Complejidad de Red)

Propósito: Evaluar el rendimiento cuando las simplificaciones implican dependencias cruzadas entre múltiples qubits, aumentando fuertemente la densidad del grafo.



Caso 3: El Muro de Ladrillos

Propósito: Medir el coste de tiempo cuando el circuito es completamente caótico y no coincide con ninguna regla del catálogo. Esto obliga al motor a revisar inútilmente todas sus equivalencias.



3. Implementación y Resultados

Para llevar a cabo las pruebas de forma automática, utilizamos una herramienta llamada pytest-benchmark. Como sabíamos que algunos circuitos complejos podían hacer colapsar el programa, le programamos una medida de seguridad usando hilos (Threading): un cronómetro interno que cancela la prueba si la computadora se queda atascada procesando por más de 5 minutos (300 segundos). Así evitamos bloqueos severos del sistema.

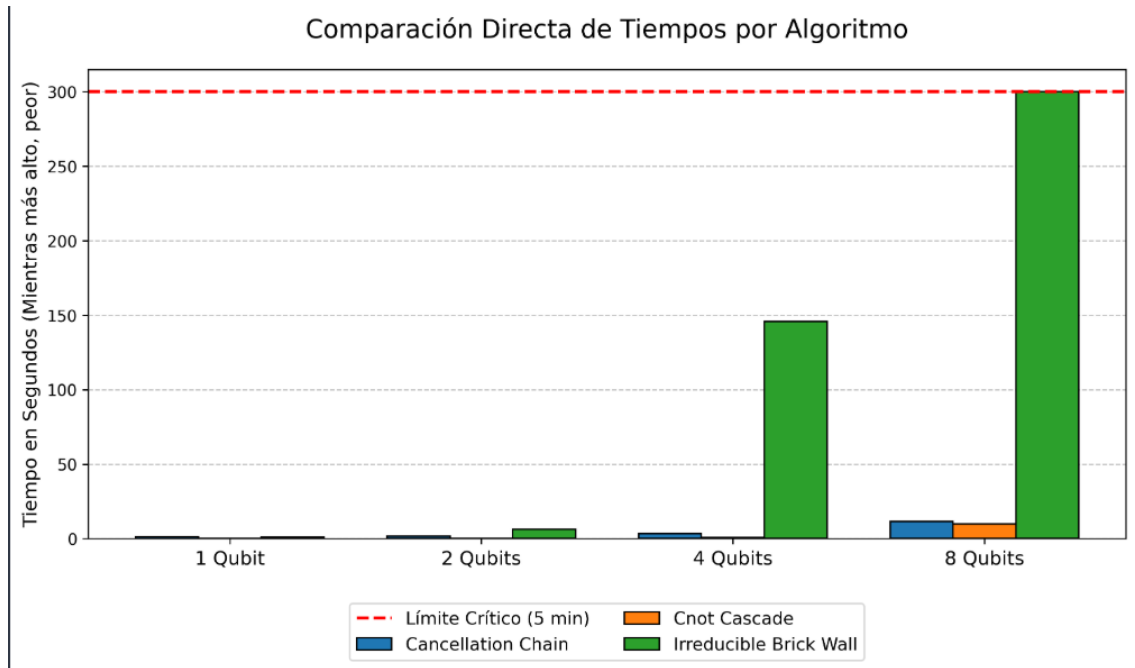
Además, dado que los casos más pesados tardan mucho en alcanzar ese límite, configuramos el sistema para que evalúe una sola pasada por cada caso (--benchmark-min-rounds=1). Si dejábamos que repitiera las pruebas largas, nos iba a consumir todo el día. Por culpa de esta restricción intencional de 1 sola ronda, los datos de tiempo mínimo, máximo y promedio (mean) que se guardan en el archivo JSON son exactamente idénticos."

Los tiempos en segundos (promediados) extraídos de "benchmark_results.json" fueron los siguientes:

Qubits	Cadenas de Anulación	Cascada CNOT	Muro de Ladrillos
1 Qubit	~1.31 segundos	-	~1.07 segundos
2 Qubits	~1.76 segundos	~0.08 segundos	~6.33 segundos
4 Qubits	~3.31 segundos	~0.97 segundos	~145.91 segundos
8 Qubits	~11.48 segundos	~9.91 segundos	> 300.00 segundos

(A tomar en cuenta: El valor omitido en el cruce de "1 Qubit" y "Cascada CNOT" es intencional. Por definición en la computación cuántica, una compuerta CNOT requiere un mínimo estricto de dos qubits para existir (uno de control y uno objetivo). Por ende, es matemáticamente imposible simular o simplificar este patrón usando un solo cable).

4. Análisis Visual e Interpretación



A partir de los datos empíricos obtenidos, se derivan las siguientes conclusiones fundamentales para el rediseño del motor de simplificación:

- Eficiencia en Patrones Conocidos:** El algoritmo de QSimplify maneja la escalabilidad de manera muy eficiente siempre y cuando los patrones del circuito coincidan con sus reglas predefinidas. En la métrica de 8 qubits, logró resolver los casos de Cadenas de Anulación y Cascadas CNOT en ~10 segundos.
- Crecimiento Exponencial Crítico:** La falla catastrófica ocurre ante los patrones asimétricos (El Muro de Ladrillos). En 4 qubits, el tiempo para comprobar que no hay simplificación salta hasta casi 150 segundos. Al intentar escalar a 8 qubits, el motor colapsa por completo, superando el límite inquebrantable de 5 minutos y entrando en un bucle ciego.
- El Cuello de Botella Matemático:** QSimplify no sufre un cuello de botella por procesar demasiadas compuertas, sino por su algoritmo de búsqueda. Pierde cantidades exponenciales de tiempo "buscando" cómo aplicar reglas en grafos complejos en lugar de abortar la misión.

5. Conclusiones y Recomendaciones Futuras

Los resultados obtenidos permitieron identificar de manera clara el comportamiento del motor de simplificación QSimplify frente a distintos niveles de complejidad. A través de los benchmarks diseñados, se comprobó que el sistema presenta un rendimiento eficiente cuando los circuitos contienen patrones conocidos y simplificables, logrando resolver incluso casos de 8 qubits en tiempos relativamente aceptables.

Sin embargo, las pruebas también evidenciaron una limitación importante en escenarios altamente asimétricos y no simplificables, representados mediante el caso “Muro de Ladrillos”. En este tipo de circuitos, el tiempo de ejecución crece de forma exponencial a medida que aumenta el número de qubits, superando el límite práctico de 5 minutos establecido para las pruebas.

El análisis realizado sugiere que el principal cuello de botella del sistema no corresponde únicamente a la cantidad de compuertas presentes en el circuito, sino principalmente al costo computacional asociado a la búsqueda de equivalencias dentro de grafos complejos. En otras palabras, el motor pierde una gran cantidad de tiempo intentando determinar si existe alguna simplificación posible antes de concluir que no la hay.

Además, durante la ejecución de los benchmarks más exigentes se observaron consumos elevados de memoria del sistema, lo que sugiere que el crecimiento exponencial podría afectar no solo el tiempo de ejecución, sino también la cantidad de estructuras internas generadas durante el proceso de búsqueda.

Como trabajo futuro, se recomienda explorar estrategias que permitan reducir el espacio de búsqueda del algoritmo, tales como:

- Implementación de heurísticas de poda.
- Detección temprana de patrones no simplificables.
- Uso de memoización o caché de subgrafos analizados previamente.
- Priorización inteligente de equivalencias más probables.
- Medición formal del consumo de memoria mediante herramientas de profiling.

Finalmente, este trabajo permitió no solo evaluar los límites actuales de QSimplify, sino también identificar de manera concreta los principales desafíos algorítmicos que deberán abordarse para mejorar la escalabilidad y viabilidad práctica del motor en circuitos cuánticos de mayor complejidad.